Experiment-6

Aim: Implement page replacement algorithms (a) FIFO (b) LRU (c) Optimal.

(a) First in First Out (FIFO)

Description: The First-In, First-Out (FIFO) page replacement algorithm is the simplest memory management method that replaces the oldest page residing in main memory the longest when a page fault occurs.

Algorithm:

1. Start
2. Initialize: Read the frame size and page sequence. Fill the memory frames array with -1 (empty). Set hit = 0, fault = 0, and a replacement pointer idx = 0.
3. Iterate: For each page in the sequence, check if it already exists in the frames:

- If Hit (Found): Increment the hit counter.
- If Fault (Not Found):
  - Insert the page into frames[idx].
  - Update the pointer circularly: idx = (idx + 1) % size.
  - Increment the fault counter.

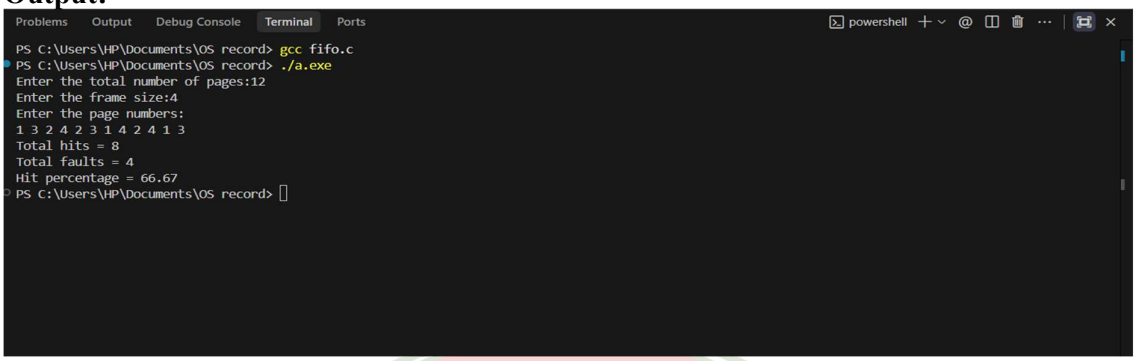
4. Output: Print the total hits, total faults, and calculate the hit percentage.
5. Stop

Code:

```
#include<stdio.h>
int main(){
    int n,size;
    printf("Enter the total number of pages:");
    scanf("%d",&n);
    int pages[n];
    printf("Enter the frame size:");
    scanf("%d",&size);
    int hit=0,fault=0;
    int frames[size];
    for(int i=0;i<size;i++){
        frames[i]=-1;
    }
    printf("Enter the page numbers:\n");
    for(int i=0;i<n;i++){
        scanf("%d",&pages[i]);
    }
    int idx = 0, check = 0;
    for(int i=0;i<n;i++){
        for(int j =0; j<size;j++){
            if(pages[i]==frames[j]){
                check = 1;
                break;
            }
        }
        if(check==0){
            frames[idx]=pages[i];
            idx=(idx+1)%size;
            fault++;
        }
        else{
            hit++;
        }
        check = 0;
    }
    printf("Total hits = %d\n",hit);
    printf("Total faults = %d\n",fault);
    printf("Hit percentage = %.2f\n",(float)hit/n*100);
```

```
    return 0;
}
```

Output:



(b) Least Recently Used (LRU)

Description: The Least Recently Used (LRU) page replacement algorithm is a memory management technique used by operating systems to decide which page to evict from physical memory (RAM) when a page fault occurs and memory is full. It operates on the principle of temporal locality, which assumes that pages heavily used in the recent past are likely to be used again in the near future.

Algorithm:

1. Start
2. Initialize: Read the frame size and page sequence. Fill memory frames with -1 (empty) and an associated time array with 0. Set hit = 0, fault = 0, and a time counter = 0.
3. Iterate: For each page in the sequence, check if it already exists in the frames:

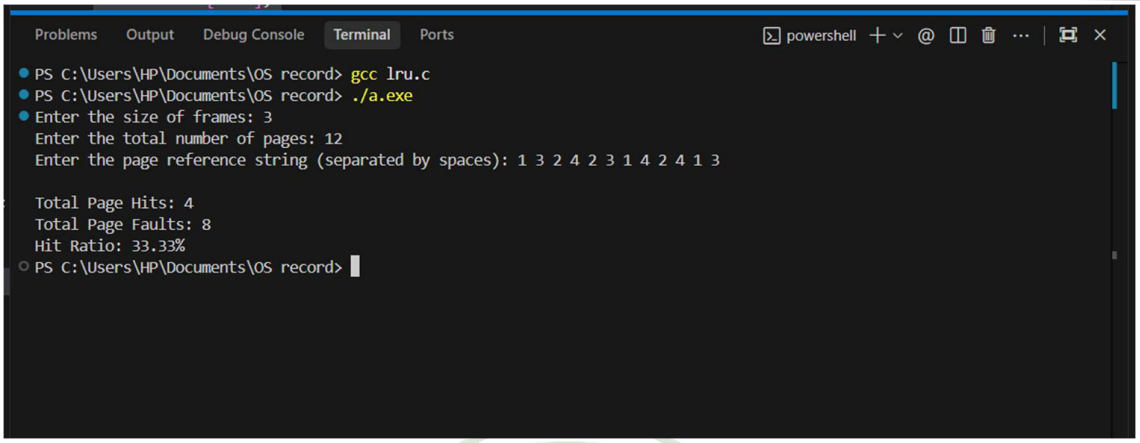
- If Hit (Found):
    - Increment the hit counter.
    - Update the found frame's access time to ++counter (marking it as most recently used).
- If Fault (Not Found):
    - Increment the fault counter.
    - If there is an empty frame, insert the new page there.
    - If frames are full, find the frame with the lowest access time (Least Recently Used) and replace it with the new page.
    - Update the new/replaced frame's access time to ++counter.

4. Output: Print the total hits, total faults, and calculate the hit percentage.
5. Stop

Code:

```
#include <stdio.h>
int main() {
    int size, n;
    printf("Enter the size of frames: ");
    scanf("%d", &size);
    printf("Enter the total number of pages: ");
    scanf("%d", &n);
    int pages[n];
    int frames[size];
    int time[size];
    for (int i = 0; i < size; i++) {
        frames[i] = -1;
        time[i] = 0;
    }
```

```
    printf("Enter the page reference string (separated by spaces): ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    int counter = 0;
    int hit = 0, fault = 0;
    for (int i = 0; i < n; i++) {
        int check = 0;
        int hit_idx = -1;
        for (int j = 0; j < size; j++) {
            if (pages[i] == frames[j]) {
                check = 1;
                hit_idx = j;
                break;
            }
        }
        if (check == 0) {
            fault++;
            int empty_idx = -1;
            for (int j = 0; j < size; j++) {
                if (frames[j] == -1) {
                    empty_idx = j;
                    break;
                }
            }
            if (empty_idx != -1) {
                frames[empty_idx] = pages[i];
                time[empty_idx] = ++counter;
            } else {
                int min_time = time[0];
                int lru_idx = 0;
                for (int j = 1; j < size; j++) {
                    if (time[j] < min_time) {
                        min_time = time[j];
                        lru_idx = j;
                    }
                }
                frames[lru_idx] = pages[i];
                time[lru_idx] = ++counter;
            }
        } else {
            hit++;
            time[hit_idx] = ++counter;
        }
    }
    printf("\nTotal Page Hits: %d\n", hit);
    printf("Total Page Faults: %d\n", fault);
    printf("Hit Ratio: %.2f%%\n", ((float)hit / n) * 100);
    return 0;
}
```



(c) Optimal Page Replacement

Description: The Optimal Page Replacement is a memory-management method that replaces the page which will not be used for the longest time in the future.

Algorithm:

1. Start.
2. Read number of frames and page reference string.
3. Initialize all frames as empty.
4. For each page:
   - If page is already in frame → Hit.
   - Otherwise → Page Fault.
5. If a frame is empty, insert the page.
6. If all frames are full, replace the page that will be used farthest in the future.
7. Calculate hit ratio and fault ratio.
8. Display results.
9. Stop.

Code:

```
#include <stdio.h>
int main() {
    int size, n;
    printf("Enter the size of frames: ");
    scanf("%d", &size);
    printf("Enter the total number of pages: ");
    scanf("%d", &n);
    int pages[n];
    int frames[size];
    for (int i = 0; i < size; i++) {
        frames[i] = -1;
    }
    printf("Enter the page reference string (separated by spaces): ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    int hit = 0, fault = 0;
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < size; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
```

```
            if (found) {
                hit++;
            } else {
                fault++;
                int empty = -1;
                for (int j = 0; j < size; j++) {
                    if (frames[j] == -1) {
                        empty = j;
                        break;
                    }
                }
                if (empty != -1) {
                    frames[empty] = pages[i];
                } else {
                    int farthest = -1;
                    int replace_idx = -1;
                    for (int j = 0; j < size; j++) {
                        int next_use = -1;
                        for (int k = i + 1; k < n; k++) {
                            if (frames[j] == pages[k]) {
                                next_use = k;
                                break;
                            }
                        }
                        if (next_use == -1) {
                            replace_idx = j;
                            break;
                        }
                        if (next_use > farthest) {
                            farthest = next_use;
                            replace_idx = j;
                        }
                    }
                    frames[replace_idx] = pages[i];
                }
            }
        }
        printf("\nTotal Page Hits: %d\n", hit);
        printf("Total Page Faults: %d\n", fault);
        printf("Hit Ratio: %.2f%%\n", ((float)hit / n) * 100);
        printf("Fault Ratio: %.2f%%\n", ((float)fault / n) * 100);
        return 0;
    }
```

```
Problems  Output  Debug Console  Terminal  Ports                                  powershell

PS C:\Users\HP\Documents\OS record> gcc optimal-page.c
PS C:\Users\HP\Documents\OS record> ./a.exe
Enter the size of frames: 3
Enter the total number of pages: 12
Enter the page reference string (separated by spaces): 1 3 2 4 2 3 1 4 2 4 1 3

Total Page Hits: 6
Total Page Faults: 6
Hit Ratio: 50.00%
Fault Ratio: 50.00%
PS C:\Users\HP\Documents\OS record>
```